

Clean Code Guidelines: Dataset Specification and Code Review Comments Framework

The Cognitive Economics of Clean Code

For an AI code reviewer, validating functional correctness through compilation and tests is only the first layer of evaluation.¹ The true value of an automated reviewer lies in its ability to minimize the "cognitive load" of future readers.³ In modern enterprise systems, code is read far more frequently than it is written.⁴ Neglecting structural design leads to technical debt, which increases exponentially and reduces a team's capacity to ship new features.⁶ During fine-tuning, the AI model must learn to quantify code complexity and suggest structural modifications that reduce it.⁷ We can evaluate the structural health of a method using McCabe's Cyclomatic Complexity formula:

$$M = E - V + 2P$$

where E represents the number of control flow edges, V represents the number of vertices (basic logic blocks), and P represents the number of exit points or connected components. While Cyclomatic Complexity measures execution paths, Cognitive Complexity measures the mental effort required to understand the control flow, penalizing nested structures and structural tricks.

Structural Quality Metrics Matrix

Smell Category	Target Boundary	Cognitive Impact	Optimal Clean Code Remedy
Method Length	10-15 lines	Lowers reader's visual and mental search strain	Extract Method refactoring
Parameter Count	<= 3 parameters	Prevents argument mismatch and complex testing	Introduce Parameter Object
Method Chaining	1 level of depth	Enforces strong	Law of Demeter

		data encapsulation	delegation
Flag Parameters	0 boolean triggers	Eliminates multi-mode conditional complexity	Method splitting (Single Purpose)

Unit 1: Meaningful Names and Intent-Revealing Abstractions

Consistent, descriptive, and unambiguous naming forms the bedrock of clean code. Identifiers must be self-documenting, making comments redundant.

Naming Conventions

- **Classes and Interfaces:** Use nouns or noun phrases in PascalCase (e.g., OrderProcessor, BillingService). Interfaces can also be adjectives describing capabilities (e.g., Runnable, Serializable).
- **Methods:** Use verbs or verb phrases in camelCase (e.g., validateOrder, calculateTax).
- **Constants:** Use SCREAMING_SNAKE_CASE (e.g., MAX_RETRY_ATTEMPTS) and replace all magic numbers with named constants to prevent cryptic code.

```
Java
// Non-Compliant: Cryptic variables, magic numbers, and redundant comments
public class OP // Bad: Cryptic class name
    // Elapsed time in days
    int d // Bad: Name reveals no intent

    public double calc(double p)
        return p * 0.15 // Bad: What is 0.15? Redundant calculation
    }
}
```

Java

```
// Compliant: Self-documenting class and variables with named constants
```

```
public final class OrderProcessor
```

```
    private static final double TAX_RATE_MULTIPLIER = 0.15 // Clear domain rule
```

```
    public double calculateAppliedTax(final double productPrice)
```

```
    {
        return productPrice * TAX_RATE_MULTIPLIER;
    }
}
```

Unit 2: The Anatomy of Clean Functions

Every method should follow the **Single Responsibility Principle (SRP)**—it must do one thing, do it well, and do it exclusively.

Single Level of Abstraction (SLA)

All statements within a method must remain at the same level of abstraction. Mixing high-level business rules with low-level buffer manipulations degrades legibility.

Command-Query Separation (CQS)

A method should be a query (returning data without modifying state) or a command (performing an action/mutation and returning void), but never both.

The Flag Argument Anti-Pattern

Passing boolean flags (e.g., `isPremium`, `formatOutput`) into a function is a clear indication that the method is performing multiple tasks. This pattern forces the caller to remember the meaning of the boolean parameter (e.g., `bookConcert(customer, true)`) and creates complex internal branching. Instead, split the behavior into distinct, well-named methods.

Java

```
// Non-Compliant: Flag arguments and multiple responsibilities inside a single method
```

```
public class ConcertService
```

```
    public Booking bookConcert(Customer customer, boolean isPremium)
```

```

        if (isPremium)
            // Expensive premium seat assignment logic
            // Payment processing
            // Notification dispatch
            return executePremiumBooking(customer)
        else
            // General admission seat assignment logic
            // Payment processing
            // Notification dispatch
            return executeStandardBooking(customer)
    }
}

```

Java

// Compliant: Separate methods express intent clearly without flag parameters

```
public class ConcertService
```

```
    private final BookingEngine bookingEngine
```

```
    private final NotificationService notificationService
```

```
    public ConcertService(BookingEngine bookingEngine, NotificationService notificationService)
```

```
    {
        this.bookingEngine = bookingEngine
```

```
        this.notificationService = notificationService
    }

```

```
    public Booking bookPremiumConcert(final Customer customer)
```

```
    {
        Booking booking = bookingEngine.allocatePremiumSeat(customer)
```

```
        notificationService.dispatchConfirmation(customer, booking)
```

```
        return booking
    }

```

```
    public Booking bookStandardConcert(final Customer customer)
```

```
    {
        Booking booking = bookingEngine.allocateStandardSeat(customer)
```

```
        notificationService.dispatchConfirmation(customer, booking)
```

```
        return booking
    }
}

```

Unit 3: Object-Oriented Integrity and the Law of Demeter

The **Law of Demeter (LoD)**, or the *Principle of Least Knowledge*, states that a module should not know about the internal details of the objects it manipulates. Objects must hide their internal structure and expose only operations rather than data.

LoD Violations (Train Wrecks): $A \rightarrow B \rightarrow C \rightarrow D$

When client code chain-calls methods on returned objects (e.g., `a.getB().getC().doSomething()`), it creates tight coupling. If any intermediate class structure changes, the client code breaks. Instead, follow the **"Tell, Don't Ask"** principle: tell the parent object what to do, and let it coordinate with its sub-objects.

Java

// Non-Compliant: Deep method chaining (Train Wrecks) violating the Law of Demeter

public class OrderService

public void displayFirstItemPrice(Customer customer)

// High coupling: OrderService is bound to Customer, Cart, Item, Product, and Money structures

Money price = customer.getShoppingCard().getItems().get(0).getProduct().getPrice()

System.out.println("Price: " + price.getAmount());

}

Java

// Compliant: Encapsulated delegation hides class layout and structures

public class OrderService

public void displayFirstItemPrice(final Customer customer)

// Enforces the "single dot" rule by talking only to immediate friends

```

    Money price = customer.getFirstCartItemPrice();
    System.out.println("Price: " + price.getAmount());
}

public class Customer
{
    private ShoppingCart shoppingCart;

    public Money getFirstCartItemPrice()
    {
        return shoppingCart.getFirstItemPrice();
    }
}

public class ShoppingCart
{
    private List<CartItem> items;

    public Money getFirstItemPrice()
    {
        if (items.isEmpty())
            return Money.ZERO;

        return items.get(0).getProductPrice();
    }
}

```

Exceptions to the Law of Demeter

- **Data Structures:** DTOs and pure data structures (classes containing data with public fields or getters but no behavior) can be chained safely, as their entire purpose is exposing data.
- **Fluent APIs / Builders:** The Builder pattern relies on method chaining to return the same or a newly modified instance of the builder, which does not violate Demeter's design goals.

Unit 4: Robust Exception Handling and Null Safety

Exceptions should be reserved strictly for truly exceptional scenarios and must never be used for program control flow.

Swallowing Exceptions

Never write empty catch blocks. Swallowing exceptions masks critical bugs, leaving the system in an unpredictable state and making debugging impossible. If an exception is truly ignorable, document the rationale in a comment and name the exception variable ignored.¹¹

Checked vs. Unchecked Exception Coupling

Avoid polluting business logic with checked exceptions. Catch lower-level exceptions and translate them into custom, unchecked runtime exceptions to maintain clean method signatures across layer boundaries.

Null Pointer Protection

Never pass null as a parameter or return null from a method. Return empty collections or use Java's Optional to handle missing values safely.

```
Java
// Non-Compliant: Catching generic Exception, swallowing bugs, and returning null
public class UserService
    private UserRepository repo;

    public User findUser(String id)
    try
        return repo.fetch(id);
    catch (Exception e) // Anti-pattern: generic catch
        // Swallowing exceptions obscures DB connection drops and syntax issues
        return null // Anti-pattern: returning null forces callers to write null checks
    }
}
```

```
Java
// Compliant: Specific exception translation with clean runtime propagation and Optional signatures
public class UserService
    private final UserRepository repo;

    public UserService(UserRepository repo)
    this.repo = Objects.requireNonNull(repo, "UserRepository dependency cannot be null")
    }
```

```

public java.util.Optional<User> findUser(final String identifier)
{
    try
    {
        return java.util.Optional.ofNullable(repo.fetch(identifier));
    }
    catch (DataAccessException e)
    {
        // Translate checked or low-level exception, preserving the original cause for debugging
        throw new UserServiceException("Failed to retrieve user record with ID: " + identifier, e);
    }
}

```

Unit 5: Automated Testing Standards (F.I.R.S.T. Principles)

To build a reliable automated testing pipeline, unit tests must adhere to the **F.I.R.S.T.** guidelines:

- **Fast:** Tests must run quickly (under 100ms per test) to encourage developers to run them frequently.
- **Independent:** Tests must not depend on the execution order or state of other tests. Every test should set up its own test data and clean up afterwards.
- **Repeatable:** Tests must return consistent results in any environment (local machines, CI/CD pipelines, offline) without external network or database dependencies.
- **Self-Validating:** Tests must have a single boolean outcome (pass or fail) using explicit assertions (e.g., assertEquals, assertTrue). No manual inspection of log files or console outputs should be required.
- **Timely:** Unit tests should be written in step with the production code, such as in Test-Driven Development (TDD) pipelines.

```

Java
// Compliant: Isolated, fast, and repeatable unit test matching the F.I.R.S.T. standard
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class AccountTest
{
    @Test
    public void deposit_ValidAmount_ShouldIncreaseBalancePercentage()
    {
        // Arrange (Set up test state)
        Account account = new Account(100.0);
    }
}

```



```

// Act (Perform test operation)
account.deposit(50.0)

// Assert (Validate expectations with clear, self-contained assertions)
assertEquals(150.0, account.getBalance(), "The balance should reflect the deposited
amount."

```

Unit 6: Custom Static Analysis Rules (PMD XPath)

The following PMD rules can be used to scan training repositories and validate code before ingestion into your dataset.¹²

1. PMD Rule: Detect Law of Demeter Violations

This rule flags chained getter sequences of three or more dots, excluding builders, fluent APIs, and native stream operations.

```

XML
<rule name="AvoidLawOfDemeterTrainWrecks" language="java" message="Avoid deep method
chaining (train wrecks) to minimize coupling." class="net.sourceforge.pmd.lang.rule.XPathRule">
  <description> identifies sequences of three or more method calls on returned references
indicating a likely Law of Demeter violation </description>
  <priority> </priority>
  <properties>
    <property name="xpath">
      <value>
        *) >= 3
        and not(PrimaryPrefix/Name
        and not(ancestor::ClassOrInterfaceDeclaration
      </value>
    </property>
  </properties>
</rule>

```

2. PMD Rule: Detect Flag / Boolean Method Parameters

This rule flags public API methods that accept primitive boolean arguments, indicating they may have multiple modes of operation.

```
XML
<rule name="AvoidBooleanFlagArguments" language="java" message="Do not use boolean flag
arguments. Split the behavior into separate methods instead."
class="net.sourceforge.pmd.lang.rule.XPathRule">
  <description> flags methods that use primitive boolean flags, suggesting a violation of the
Single Responsibility Principle </description>
  <priority> </priority>
  <properties>
    <property name="xpath">
      <value>
        /MethodDeclarator/FormalParameters/FormalParameter
        /Type/PrimitiveType[@Image = 'boolean']
      </value>
    </property>
  </properties>
</rule>
```

Unit 7: Master Dataset Schema Specification (JSONL Format)

The master dataset should be structured as a single-line JSON Lines (JSONL) file to support efficient parsing during training.

```
JSON
{
  "instance_id" "MSR_CLEAN_2026_LOD_023"
  "metadata"
  "problem_domain" "Design Quality & Coupling"
  "difficulty" "Hard"
```

```

"targeted_smell" "Law of Demeter Violation"
"pmd_rule_reference" "category/java/bestpractices.xml/AvoidLawOfDemeterTrainWrecks"
"context"
"repository" "apache/commons-collections"
"jdk_version" "Java 21"
"problem_statement" "The OrderScheduler class directly accesses nested collections and
pricing properties across several classes, creating deep structural coupling."
"original_code" "package org.apache.commons.collections;\n\npublic class OrderScheduler {\n
public double calculateScheduledPrice(Customer customer) {\n    // High coupling: breaking
encapsulation by reaching deep into cart layout\n    return
customer.getShoppingCart().getCartDetails().getItemList().get(0).getPricing().getDiscountedAmou
nt();\n } \n}"
"refactored_code" "package org.apache.commons.collections;\n\npublic class OrderScheduler {\n
public double calculateScheduledPrice(final Customer customer) {\n    // Low coupling: delegates
responsibility to the Customer class\n    return
customer.getPrimaryScheduledItemDiscountedPrice();\n } \n}"
"structured_review"
"severity" "Major"
"line_range" 9 3
"diagnostic" "The calculateScheduledPrice method violates the Law of Demeter by chaining
method calls on returned sub-objects."
"remediation_justification" "Chaining method calls through getShoppingCart(), getCartDetails(),
and getPricing() creates tight structural coupling across six classes. Refactoring to
customer.getPrimaryScheduledItemDiscountedPrice() delegates the task, protecting the scheduler
from changes to the internal structures of Cart and Product."

```

JSON

```

"instance_id" "MSR_CLEAN_2026_FLAG_091"
"metadata"
"problem_domain" "Method Design"
"difficulty" "Medium"

```

```

"targeted_smell" "Boolean Flag Argument"
"pmd_rule_reference" "category/java/bestpractices.xml/AvoidBooleanFlagArguments"
"context"
"repository" "spring-projects/spring-boot"
"jdk_version" "Java 17"
"problem_statement" "The FileExporter class accepts a boolean flag to determine whether to
write CSV or JSON outputs, violating the Single Responsibility Principle."
"original_code" "package com.spring.boot.export;\n\nimport java.util.List;\n\npublic class
FileExporter {\n    public void exportData(List<String> records, boolean isJson) {\n        if (isJson) {\n
// complex JSON serialization and writing logic\n        } else {\n            // complex CSV construction
and writing logic\n        }\n    }\n}"
"refactored_code" "package com.spring.boot.export;\n\nimport java.util.List;\n\npublic class
FileExporter {\n    public void exportAsJson(final List<String> records) {\n        // encapsulated JSON
serialization and writing logic\n    }\n\n    public void exportAsCsv(final List<String> records) {\n        //
encapsulated CSV construction and writing logic\n    }\n}"
"structured_review"
"severity" "Medium"
"line_range" "3 6"
"diagnostic" "The method exportData uses a boolean flag parameter 'isJson' to execute
different serialization logic."
"remediation_justification" "Passing a boolean flag indicates that the method has multiple
responsibilities and execution modes. Splitting it into two distinct methods, exportAsJson and
exportAsCsv, improves readability and simplifies testing for both formatters."

```

JSON

```

"instance_id" "MSR_CLEAN_2026_EX_044"
"metadata"
"problem_domain" "Error & Exception Handling"
"difficulty" "Hard"
"targeted_smell" "Swallowed Exception & Generic Catch"
"pmd_rule_reference" "category/java/errorprone.xml/EmptyCatchBlock"

```

```

"context"
"repository" "eclipse/jetty.project"
"jdk_version" "Java 21"
"problem_statement" "The server configuration loader catches all exceptions silently, making
system errors and socket connection drops impossible to debug."

"original_code" "package org.eclipse.jetty.config;\n\nimport java.io.*;\n\npublic class
ServerConfigLoader {\n  public Properties loadProperties(String path) {\n    Properties props =
new Properties();\n    try {\n      props.load(new FileInputStream(path));\n    } catch (Exception
e) {\n      // Swallowing exceptions makes diagnosing issues impossible\n    }\n    return
props;\n  }\n}"
"refactored_code" "package org.eclipse.jetty.config;\n\nimport java.io.*;\n\npublic class
ServerConfigLoader {\n  public java.util.Properties loadProperties(final String path) {\n
java.util.Properties props = new java.util.Properties();\n    try (FileInputStream fis = new
FileInputStream(path)) {\n      props.load(fis);\n    } catch (FileNotFoundException e) {\n
throw new ConfigurationException(\"Configuration file not found on path: \" + path, e);\n    } catch
(IOException e) {\n      throw new ConfigurationException(\"Failed to load or parse server
configuration properties on path: \" + path, e);\n    }\n    return props;\n  }\n}"
"structural_review"
"severity" "Critical"
"line_range" 10 6
"diagnostic" "The catch block catches generic 'Exception' and swallows it completely without
logging or throwing."
"remediation_justification" "Swallowing generic Exception blocks masks critical system errors
(like socket exhaustion, file lockouts, or IO failures). Catching specific exceptions and translating
them into custom unchecked runtime exceptions preserves error context and simplifies
debugging."

```

Works cited

1. [2603.00821] Taking a Closer Look at Warnings Generated by PMD and SonarQube, their Rules and Compliance to Established Coding Standards - arXiv, accessed on June 12, 2026, <https://arxiv.org/abs/2603.00821>
2. Source code review: A comprehensive guide to secure development - Sonar, accessed on June 12, 2026, <https://www.sonarsource.com/resources/library/source-code-review/>
3. Home · microsoft/CodeXGLUE Wiki · GitHub, accessed on June 12, 2026, <https://github.com/microsoft/CodeXGLUE/wiki>
4. Bad Java code in 7 easy steps - Andreas Maier - Medium, accessed on June 12, 2026,

- <https://astromaier.medium.com/bad-java-code-in-7-easy-steps-792eee03855b>
5. The Unseen Memory Leak: How ThreadLocal Variables Can Bring Down Your Application, accessed on June 12, 2026,
<https://blog.heaphero.io/threadlocal-memory-leak/>
 6. Make your Java code smell nice and fresh - freeCodeCamp, accessed on June 12, 2026,
<https://www.freecodecamp.org/news/do-not-allow-bad-smells-in-your-java-code-4e8ad244393/>
 7. Java Exception Error Enumerations Anti-pattern - Software Engineering Stack Exchange, accessed on June 12, 2026,
<https://softwareengineering.stackexchange.com/questions/444232/java-exception-error-enumerations-anti-pattern>
 8. CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation - arXiv, accessed on June 12, 2026,
<https://arxiv.org/pdf/2102.04664>
 9. learning-notes/books/effective-java.md at master · keyvanakbary ..., accessed on June 12, 2026,
<https://github.com/keyvanakbary/learning-notes/blob/master/books/effective-java.md>
 10. Writing Custom Pmd Rules, accessed on June 12, 2026,
<https://sites.google.com/view/writing-custom-pmd-rules-52/>